

---

**Batch 14 — Cashier Payment Screen + Receipt Flow**

---

Daniel Macharia <danielmacharia43@gmail.com>  
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:15

## Batch 14 — Cashier Payment Screen + Receipt Flow

**Meat Lovers CIMS powered by YohPal**

### 14A. Update

**apps/pos-pwa/src/app/routes.jsx**

Add imports:

```
import CashierPaymentPage from '../features/cashier/pages/CashierPaymentPage'
import ReceiptPage from '../features/cashier/pages/ReceiptPage'
import PrintQueuePage from '../features/cashier/pages/PrintQueuePage'
```

Add routes inside PosLayout:

```
<Route path="/cashier" element={<CashierPaymentPage />} />
<Route path="/receipt/:orderId" element={<ReceiptPage />} />
<Route path="/print-queue" element={<PrintQueuePage />} />
```

---

### 14B. Update

**apps/pos-pwa/src/components/common/MobileNav.jsx**

Add cashier link:

```
<Link to="/cashier">Cashier</Link>
```

---

### 14C.

**apps/pos-pwa/src/features/cashier/api/cashierApi.js**

import apiClient from '../../../lib/apiClient'

```
export const cashierApi = {
  orders: async () => {
    const { data } = await apiClient.get('/orders')
    return data.data.orders
  },
  cashSettle: async (orderId, payload) => {
    const { data } = await apiClient.post(`/payments/cash-settle/${orderId}`, payload)
    return data
  },
  mpesaPending: async (orderId, payload) => {
    const { data } = await apiClient.post(`/payments/mpesa-pending/${orderId}`, payload)
    return data
  },
  receipt: async (orderId) => {
    const { data } = await apiClient.get(`/payments/receipt/${orderId}`)
    return data.data
  },
  printQueue: async () => {
    const { data } = await apiClient.get('/payments/print-queue')
    return data.data.print_queue
  },
  audit: async () => {
    const { data } = await apiClient.get('/payments/audit')
    return data.data.payment_audit
  },
}
```

## 14D.

**apps/pos-pwa/src/features/cashier/pages/CashierPaymentPage.jsx**

```
import React, { useMemo, useState } from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import { Link } from 'react-router-dom'
import { cashierApi } from '../api/cashierApi'

export default function CashierPaymentPage() {
  const qc = useQueryClient()
  const [search, setSearch] = useState('')
  const [status, setStatus] = useState('')

  const { data, isLoading, error } = useQuery({
    queryKey: ['cashier-orders'],
    queryFn: cashierApi.orders,
  })

  const cashMutation = useMutation({
    mutationFn: ({ orderId, amount }) => cashierApi.cashSettle(orderId, { amount }),
    onSuccess: () => {
      setStatus('Cash payment settled successfully.')
      qc.invalidateQueries({ queryKey: ['cashier-orders'] })
    },
  })

  const mpesaMutation = useMutation({
    mutationFn: ({ orderId, phone }) => cashierApi.mpesaPending(orderId, { phone }),
    onSuccess: () => {
      setStatus('M-Pesa payment marked as pending.')
      qc.invalidateQueries({ queryKey: ['cashier-orders'] })
    },
  })

  const orders = useMemo(() => {
    const list = data || []
    return list.filter((order) => {
      const text = `${order.order_number} ${order.table_number} ${order.order_status}`.toLowerCase()
      return text.includes(search.toLowerCase())
    })
  }, [data, search])

  return (
    <main>
      <h1>Cashier Payments</h1>
      <p>Search orders, settle cash, or mark M-Pesa as pending.</p>

      {status ? <p>{status}</p> : null}
      {isLoading ? <p>Loading orders...</p> : null}
      {error ? <p>Could not load orders.</p> : null}
      {cashMutation.error ? <p>Cash settlement failed.</p> : null}
      {mpesaMutation.error ? <p>M-Pesa pending update failed.</p> : null}

      <input
        placeholder="Search by order number, table, or status"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        style={inputStyle}
      />

      <div style={{ display: 'grid', gap: 12, marginTop: 16 }}>
        {orders.map((order) => (
          <OrderPaymentCard
            key={order.id}
            order={order}
            onCash={(amount) => cashMutation.mutate({ orderId: order.id, amount })}
            onMpesa={(phone) => mpesaMutation.mutate({ orderId: order.id, phone })}
          />
        ))}
      </div>
    </main>
  )
}

function OrderPaymentCard({ order, onCash, onMpesa }) {
  const [amount, setAmount] = useState(order.total_amount || '')
  const [phone, setPhone] = useState('')

  return (
    <section style={cardStyle}>
      <strong>{order.order_number}</strong>
      <p>Table: {order.table_number}</p>
      <p>Status: {order.order_status}</p>
    </section>
  )
}
```

```

    <p>Total: KES {order.total_amount}</p>

    <input
      placeholder="Cash amount received"
      value={amount}
      onChange={(e) => setAmount(e.target.value)}
      style={inputStyle}
    />

    <button onClick={() => onCash(Number(amount))} style={buttonStyle}>
      Settle Cash
    </button>

    <input
      placeholder="Customer M-Pesa phone"
      value={phone}
      onChange={(e) => setPhone(e.target.value)}
      style={inputStyle}
    />

    <button onClick={() => onMpesa(phone)} style={secondaryButton}>
      Mark M-Pesa Pending
    </button>

    <Link to={`\receipt/${order.id}`} style={receiptLink}>
      View Receipt
    </Link>
  </section>
)
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
  display: 'grid',
  gap: 8,
}

const inputStyle = {
  width: '100%',
  padding: 14,
  borderRadius: 12,
  border: '1px solid #d1d5db',
}

const buttonStyle = {
  padding: 14,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: 'fff',
}

const secondaryButton = {
  padding: 14,
  borderRadius: 12,
  border: '1px solid #111827',
  background: 'fff',
  color: '#111827',
}

const receiptLink = {
  padding: 12,
  borderRadius: 12,
  background: '#f3f4f6',
  textAlign: 'center',
}

```

## 14E.

**apps/pos-pwa/src/features/cashier/pages/ReceiptPage.jsx**

```

import React from 'react'
import { useParams } from 'react-router-dom'
import { useQuery } from '@tanstack/react-query'
import { cashierApi } from '../api/cashierApi'

export default function ReceiptPage() {
  const { orderId } = useParams()

  const { data, isLoading, error } = useQuery({
    queryKey: ['receipt', orderId],

```

```

    queryFn: () => cashierApi.receipt(orderId),
  })

  return (
    <main>
      <h1>Receipt</h1>

      {isLoading ? <p>Loading receipt...</p> : null}
      {error ? <p>Could not load receipt.</p> : null}

      {data ? (
        <section style={receiptStyle}>
          <h2>Meat Lovers</h2>
          <p>Powered by YohPal</p>

          <hr />

          <p>Order: {data.order.order_number}</p>
          <p>Table: {data.order.table_number}</p>
          <p>Status: {data.order.order_status}</p>

          <hr />

          {(data.items || []).map((item) => (
            <div key={item.id} style={{ display: 'flex', justifyContent: 'space-between' }}>
              <span>{item.product_name} x {item.quantity}</span>
              <span>KES {item.total_price}</span>
            </div>
          ))}

          <hr />

          <h3>Total: KES {data.order.total_amount}</h3>

          <h4>Payments</h4>
          {(data.payments || []).map((payment) => (
            <p key={payment.id}>
              {payment.payment_method}: KES {payment.amount} - {payment.payment_status}
            </p>
          ))}

          <button onClick={() => window.print()} style={buttonStyle}>
            Print Receipt
          </button>
        </section>
      ) : null}
    </main>
  )
}

const receiptStyle = {
  background: '#fff',
  padding: 18,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
}

const buttonStyle = {
  width: '100%',
  marginTop: 16,
  padding: 14,
  borderRadius: 12,
  border: 'none',
  background: '#111827',
  color: '#fff',
}

```

## 14F.

**apps/pos-pwa/src/features/cashier/pages/PrintQueuePage.jsx**

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import { cashierApi } from '../api/cashierApi'

export default function PrintQueuePage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['print-queue'],
    queryFn: cashierApi.printQueue,
  })

  return (
    <main>
      <h1>Print Queue</h1>

```

```

    <p>Kitchen, bar, and receipt print queue placeholder.</p>

    {isLoading ? <p>Loading print queue...</p> : null}
    {error ? <p>Could not load print queue.</p> : null}

    <div style={{ display: 'grid', gap: 12 }}>
      {(data || []).map((item) => (
        <section key={item.id} style={cardStyle}>
          <strong>{item.print_type}</strong>
          <p>Order: {item.order_number}</p>
          <p>Status: {item.print_status}</p>
          <p>Created: {item.created_at}</p>
        </section>
      ))}
    </div>
  </main>
)
}

const cardStyle = {
  background: '#fff',
  padding: 14,
  borderRadius: 14,
  border: '1px solid #e5e7eb',
}

```

---

## 14G.

**database/migrations/023\_print\_queue.sql**

```

CREATE TABLE print_queue (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  order_id BIGINT NOT NULL,

  print_type ENUM(
    'KITCHEN',
    'BAR',
    'CUSTOMER_RECEIPT'
  ) NOT NULL,

  print_status ENUM(
    'PENDING',
    'PRINTED',
    'FAILED'
  ) DEFAULT 'PENDING',

  printed_at TIMESTAMP NULL,

  FOREIGN KEY (order_id)
    REFERENCES orders(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

---

## 14H. Update

**backend/routes/api.php**

Add routes:

```

$router->post('/api/payments/mpesa-pending/{id}', [PaymentsController::class, 'mpesaPending']);
$router->get('/api/payments/receipt/{id}', [PaymentsController::class, 'receipt']);
$router->get('/api/payments/print-queue', [PaymentsController::class, 'printQueue']);
$router->get('/api/payments/audit', [PaymentsController::class, 'audit']);

```

---

## 14I. Update

**backend/app/Controllers/PaymentsController.php**

Add these methods inside the class:

```

public function mpesaPending(int $orderId): void
{
    $user = Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'CASHIER'
    ]);
}

```

```

    });

    $data = Request::body();
    $db = Database::connection();

    $stmt = $db->prepare(
        'INSERT INTO payments
        (order_id, payment_method, amount, transaction_reference, payment_status)
        VALUES (:order_id, :method, :amount, :reference, :status)'
    );

    $stmt->execute([
        'order_id' => $orderId,
        'method' => 'MPESA',
        'amount' => $data['amount'] ?? 0,
        'reference' => 'MPESA-PENDING-' . $orderId . '-' . time(),
        'status' => 'PENDING',
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'PAYMENTS',
        'MPESA_PENDING',
        $orderId,
        null,
        $data
    );

    Response::success([], 'M-Pesa payment marked as pending');
}

public function receipt(int $orderId): void
{
    Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'CASHIER',
        'WAITER'
    ]);

    $db = Database::connection();

    $orderStmt = $db->prepare('SELECT * FROM orders WHERE id = :id LIMIT 1');
    $orderStmt->execute(['id' => $orderId]);
    $order = $orderStmt->fetch();

    if (!$order) {
        Response::error('Order not found', 404);
    }

    $itemsStmt = $db->prepare(
        'SELECT
            oi.*,
            p.product_name
        FROM order_items oi
        JOIN products p ON p.id = oi.product_id
        WHERE oi.order_id = :order_id'
    );

    $itemsStmt->execute(['order_id' => $orderId]);

    $paymentsStmt = $db->prepare(
        'SELECT * FROM payments WHERE order_id = :order_id ORDER BY created_at DESC'
    );

    $paymentsStmt->execute(['order_id' => $orderId]);

    Response::success([
        'order' => $order,
        'items' => $itemsStmt->fetchAll(),
        'payments' => $paymentsStmt->fetchAll(),
    ]);
}

public function printQueue(): void
{
    Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'CASHIER'
    ]);

    $stmt = Database::connection()->query(
        'SELECT
            pq.*,'

```

```

        o.order_number
    FROM print_queue pq
    JOIN orders o ON o.id = pq.order_id
    ORDER BY pq.created_at DESC'
);

Response::success(['print_queue' => $stmt->fetchAll()]);
}

public function audit(): void
{
    Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'CASHIER',
        'ACCOUNTANT'
    ]);

    $stmt = Database::connection()->query(
        "SELECT
            al.*,
            u.full_name
        FROM audit_logs al
        LEFT JOIN users u ON u.id = al.user_id
        WHERE al.module_name = 'PAYMENTS'
        ORDER BY al.created_at DESC"
    );

    Response::success(['payment_audit' => $stmt->fetchAll()]);
}

```

---

## 14J. Update

### backend/app/Controllers/OrdersController.php

Inside store(), after order items are inserted and before \$db->commit();, add:

```

$printTypes = ['KITCHEN'];

foreach ($data['items'] as $item) {
    $categoryStmt = $db->prepare(
        'SELECT product_category FROM products WHERE id = :id LIMIT 1'
    );

    $categoryStmt->execute(['id' => $item['product_id']]);
    $product = $categoryStmt->fetch();

    if ($product && $product['product_category'] === 'ALCOHOLIC_DRINK') {
        $printTypes[] = 'BAR';
    }
}

foreach (array_unique($printTypes) as $printType) {
    $printStmt = $db->prepare(
        'INSERT INTO print_queue
        (order_id, print_type, print_status)
        VALUES (:order_id, :print_type, "PENDING")'
    );

    $printStmt->execute([
        'order_id' => $orderId,
        'print_type' => $printType,
    ]);
}

```

---

## 14K. Update

### backend/app/Controllers/PaymentsController.php

Inside cashSettle(), before final response, add:

```

$printStmt = $db->prepare(
    'INSERT INTO print_queue
    (order_id, print_type, print_status)
    VALUES (:order_id, "CUSTOMER_RECEIPT", "PENDING")'
);

$printStmt->execute([
    'order_id' => $orderId,
]);

```

---

## 14L.

docs/CASHIER\_PAYMENT\_RECEIPT\_FLOW.md

# Batch 14 – Cashier Payment Screen + Receipt Flow

## System

Meat Lovers CIMS powered by YohPal

## Purpose

This batch adds cashier payment control and receipt visibility.

## Screens Added

1. Cashier payment page
2. Order search
3. Cash settlement
4. M-Pesa pending placeholder
5. Paid receipt view
6. Print queue placeholder

## Backend Added

1. M-Pesa pending endpoint
2. Receipt endpoint
3. Print queue endpoint
4. Payment audit endpoint

## Print Queue Types

- KITCHEN
- BAR
- CUSTOMER\_RECEIPT

## Control Rules

### Cash Payment

Cash settlement changes order status to:

```text  
PAID

### M-Pesa

M-Pesa is currently only marked as:

PENDING

Real Daraja callback wiring must come later.

### Receipt

Receipt shows:

- order number
- table
- status
- items
- total amount
- payment records

### Audit

All payment-sensitive actions must be audit logged.

---

# Batch 14 Outcome

Cashier flow now includes:

- ✓ cashier mobile/payment page
- ✓ order search
- ✓ cash settlement
- ✓ M-Pesa pending placeholder
- ✓ paid receipt view
- ✓ kitchen/bar receipt print queue placeholder

✅ payment audit visibility

# Next Smart Move

Build \*\*Batch 15 – Monitoring Dashboard Live Control UI\*\*, including:

- owner dashboard
- sales summary cards
- low stock alerts
- pending approvals
- pending M-Pesa
- unsold food alerts
- kitchen queue count
- bar low-stock count
- cashier settlement count